

## SIMATS ENGINEERING ASSESSMENT TOOL 2

**COURSE CODE:** MLA03

**COURSE NAME:** Reinforcement learning for Environment and Agent

---

### COURSE OUTCOME COVERED

**CO3:** *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.( BL2, BL3, BL6)*

*Assessment Tool Weightage: 35%*

---

Dr G. A. SENTHIL

**QUESTIONS: 10**

**Total Marks: 50**

Annexure A

**Experiments**

**Duration: 3hrs**

---

### ***Code of Conduct:***

*I ALLAPAREDDY JAHNAVI (192472214) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student: A. Jahnavi***

# **1)Design and develop a Policy Gradient-based Reinforcement Learning model for an Intelligent Traffic Signal Control System.**

## **Problem Statement**

Urban traffic congestion leads to increased waiting time, fuel consumption, and pollution. A Reinforcement Learning agent can dynamically control traffic signals by learning the optimal signal timing from traffic conditions.

## **Objective**

- Minimize average vehicle waiting time.
- Maximize traffic throughput.
- Compare REINFORCE and A2C algorithms.
- Improve convergence speed.

## **Environment Setup**

- Python 3.11
- TensorFlow 2.x
- NumPy
- Matplotlib
- OpenAI Gym (Custom Traffic Environment)

## **State Space**

The state contains:

- Number of vehicles in North lane
- Number of vehicles in South lane
- Number of vehicles in East lane
- Number of vehicles in West lane
- Current traffic light phase

## **Example:**

State = [12, 8, 15, 6, Green\_NS]

## **Action Space**

The agent selects one action:

- Keep North-South Green
- Keep East-West Green
- Switch Signal

## Reward Function

Reward =  
+ Vehicles Passed  
– Waiting Time  
– Queue Length

## Example

Reward = +20 – 8 = +12

## Policy Gradient Algorithm

### REINFORCE

- Monte Carlo method
- Updates policy after one complete episode
- Higher variance

### A2C

- Actor learns policy
- Critic estimates value function
- Lower variance
- Faster convergence

## Implementation Steps

1. Create traffic simulation environment.
2. Initialize policy network.
3. Define state and action spaces.
4. Select REINFORCE and A2C models.
5. Train both models.
6. Update network weights.
7. Monitor cumulative rewards.
8. Evaluate waiting time and throughput.
9. Compare convergence.
10. Record observations.

## Evaluation Metrics

- Average Waiting Time
- Throughput
- Episode Reward
- Convergence Rate

## Comparison

| Metric       | REINFORCE | A2C   |
|--------------|-----------|-------|
| Waiting Time | Higher    | Lower |
| Throughput   | Moderate  | High  |
| Convergence  | Slow      | Fast  |
| Stability    | Medium    | High  |

## Results Analysis

A2C significantly reduces vehicle waiting time due to the Critic network, which provides better value estimation. REINFORCE is simpler but converges slowly.

## Conclusion

A2C outperforms REINFORCE in intelligent traffic signal control by achieving lower waiting time, higher throughput, and faster convergence.

## 2)Design and implement an Actor-Critic (A2C/A3C) model for an Autonomous Warehouse Robot.

### Problem Statement

An autonomous warehouse robot must efficiently collect and deliver packages while avoiding obstacles and minimizing travel time.

### Objective

- Maximize package delivery.
- Reduce travel distance.
- Compare Vanilla Policy Gradient and Actor-Critic.

### Environment Setup

- Python
- TensorFlow
- NumPy
- Custom Warehouse Environment

### State Space

- Robot Position
- Package Location
- Destination
- Battery Level
- Obstacles

## **Example**

State = [4,5,2,8,80%]

## **Action Space**

- Move Forward
- Turn Left
- Turn Right
- Pick Package
- Deliver Package

## **Reward Function**

+100 Delivery

−5 Collision

−1 Each Move

## **Algorithm**

Vanilla Policy Gradient

- Learns only policy.
- High variance.

## **Actor-Critic**

- Actor
- Chooses actions.
- Critic
- Evaluates actions.

## **Implementation Steps**

1. Create warehouse environment.
2. Initialize Actor-Critic networks.
3. Observe state.
4. Select action.
5. Execute action.
6. Receive reward.
7. Update Actor.
8. Update Critic.
9. Repeat training.
10. Evaluate.

## **Evaluation Metrics**

- Total Reward
- Task Completion Time
- Training Stability

**Comparison**

| Metric        | Policy Gradient | Actor-Critic |
|---------------|-----------------|--------------|
| Reward        | Medium          | High         |
| Stability     | Low             | High         |
| Training Time | High            | Low          |

**Results Analysis**

Actor-Critic converges faster because the Critic guides policy updates, reducing variance.

**Conclusion**

Actor-Critic provides faster learning and higher delivery efficiency for warehouse automation.

**3)Develop a PPO Model for Smart HVAC Energy Management.**

**Problem Statement**

Smart buildings require intelligent HVAC control to reduce energy consumption while maintaining occupant comfort.

**Objective**

- Minimize electricity usage.
- Maintain indoor comfort.
- Compare PPO with REINFORCE.

**Environment Setup**

- Python
- TensorFlow
- NumPy
- Smart HVAC Simulator

**State Space**

- Indoor Temperature
- Outdoor Temperature
- Humidity
- Occupancy
- Energy Consumption

## Action Space

- Increase Cooling
- Decrease Cooling
- Increase Heating
- Decrease Heating

## Reward Function

Reward =  
Comfort Score – Energy Cost

## PPO Algorithm

PPO limits policy updates using clipped objectives, improving training stability.

## Implementation Steps

1. Build HVAC simulator.
2. Define PPO network.
3. Observe environment.
4. Select HVAC action.
5. Calculate reward.
6. Store transitions.
7. Update policy.
8. Repeat.
9. Monitor learning.
10. Compare with REINFORCE.

## Evaluation Metrics

- Energy Consumption
- Comfort Index
- Episode Reward

## Comparison

| Metric        | PPO  | REINFORCE |
|---------------|------|-----------|
| Energy Saving | High | Medium    |
| Stability     | High | Medium    |
| Convergence   | Fast | Slow      |

## Results Analysis

PPO achieves lower energy consumption while maintaining comfort due to stable policy updates.

## **Conclusion**

PPO is better suited for smart HVAC systems because it balances energy efficiency and occupant comfort.

## **4)Develop a Deep Reinforcement Learning Model using PPO for Autonomous Drone Navigation.**

### **Problem Statement**

An autonomous drone must navigate safely in dynamic environments while avoiding obstacles and minimizing energy consumption.

### **Objective**

- Improve navigation accuracy.
- Reduce collisions.
- Optimize battery usage.

### **Environment Setup**

- Python
- TensorFlow
- OpenAI Gym
- Drone Simulation

### **State Space**

- Drone Position
- Velocity
- Obstacle Distance
- Battery Level

### **Action Space**

- Move Forward
- Turn Left
- Turn Right
- Ascend
- Descend

### **Reward Function**

+ Reach Goal

– Collision

– Energy Consumption

## **PPO Algorithm**

Policy network learns the best navigation strategy while maintaining stable updates.

## **Implementation Steps**

1. Build drone simulator.
2. Initialize PPO.
3. Observe state.
4. Select action.
5. Receive reward.
6. Update policy.
7. Continue training.
8. Monitor performance.
9. Evaluate.
10. Analyze.

## **Evaluation Metrics**

- Navigation Accuracy
- Obstacle Avoidance
- Energy Consumption
- Policy Convergence

## **Results Analysis**

The PPO agent successfully avoids obstacles and reaches the destination with lower energy consumption than baseline methods.

## **Conclusion**

PPO provides reliable navigation and stable learning for autonomous drone applications.

## **5)Design, Implement, and Evaluate a PPO Controller for Autonomous Lane Keeping.**

### **Problem Statement**

An autonomous vehicle must maintain its lane while ensuring passenger safety and minimizing steering errors.

### **Objective**

- Reduce lane deviation.
- Improve driving safety.
- Compare PPO with A2C.

## **Environment Setup**

- Python
- TensorFlow
- CARLA Simulator
- NumPy

## **State Space**

- Vehicle Position
- Lane Offset
- Steering Angle
- Speed

## **Action Space**

- Steer Left
- Steer Right
- Accelerate
- Brake

## **Reward Function**

+ Stay in Lane

– Lane Departure

– Collision

## **PPO Algorithm**

The PPO agent updates its policy using clipped probability ratios to maintain stable learning.

## **Implementation Steps**

1. Create lane-keeping simulator.
2. Initialize PPO network.
3. Observe driving state.
4. Predict steering action.
5. Execute action.
6. Receive reward.
7. Update network.
8. Continue training.
9. Compare with A2C.
10. Record results.

Evaluation Metrics

- Lane Deviation
- Safety Score
- Reward Convergence
- Training Efficiency

Comparison

| Metric         | PPO  | A2C      |
|----------------|------|----------|
| Lane Deviation | Low  | Medium   |
| Safety         | High | Medium   |
| Stability      | High | Good     |
| Convergence    | Fast | Moderate |

Results Analysis

PPO maintains smoother steering control and fewer lane departures compared to A2C due to its clipped objective function, resulting in more stable policy updates.

Conclusion

PPO is an effective policy-based reinforcement learning algorithm for autonomous lane-keeping, providing improved safety, faster convergence, and better overall driving performance compared to A2C.

## RUBRICS FOR CALCULATION

| Criteria                  | Weightage | Level 4 – Exemplary                                                                           | Level 3 – Proficient                                        | Level 2 – Developing                                | Level 1 – Beginning                              |
|---------------------------|-----------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------|
| Understanding of Concepts | 25        | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design       | 30        | Well-planned, systematic implementation with optimal solution                                 | Correct implementation with minor issues                    | Basic implementation with errors or inefficiencies  | Incorrect or incomplete implementation           |
| Use of Tools              | 20        | Effective and advanced use of tools/technologies with proper configuration                    | Appropriate use of tools with correct execution             | Limited or inconsistent use of tools                | Incorrect or minimal use of tools                |
| Analysis of Results       | 15        | Insightful analysis with accurate interpretation and validation of results                    | Good analysis with reasonable interpretation                | Basic analysis with limited insights                | Poor or incorrect analysis of results            |
| Documentation             | 10        | Clear, well-structured documentation with diagrams, code, and results                         | Organized documentation with necessary details              | Basic documentation with missing details            | Poor or incomplete documentation                 |